

Gov 51 - PSet 3

Tyler Dang

<https://github.com/tylerdang25/Gov-51---PSet-3>

1. Setup And Data Exploration

Load Packages and Data

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.6
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.1      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.2
v purrr      1.2.0
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(glmnet)
```

Loading required package: Matrix

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Loaded glmnet 4.1-10

```
library(knitr)
```

```
compas <- read.csv("data/raw/compas_clean.csv")
```

Observations and Variables? Unit?

```
dim(compas)
```

```
[1] 7214  17
```

There are 7214 observations in the Compas dataset and 17 variables.

Examine the Outcome Variable

```
mean(compas$recidivism)
```

```
[1] 0.4506515
```

The recidivism rate is .451.

The outcome is binary as it takes only two values: 0 or 1.

This matters for linear models because a linear probability model produces predictions as probabilities of recidivism, but linear models can produce predictions below 0 or above 1 which are not reasonable as probabilities, but rather a change in probability.

Summary Statistics

```
summary_table <- data.frame(variable = names(compas),  
                             mean = sapply(compas, mean, na.rm = TRUE),  
                             sd   = sapply(compas, sd, na.rm = TRUE))  
kable(summary_table, digits = 3)
```

	variable	mean	sd
recidivism	recidivism	0.451	0.498
age	age	34.818	11.889
female	female	0.193	0.395
black	black	0.512	0.500
hispanic	hispanic	0.088	0.284
asian	asian	0.004	0.066
other_race	other_race	0.052	0.223
native_american	native_american	0.002	0.050
priors_count	priors_count	3.472	4.883
felony	felony	0.647	0.478
juv_fel_count	juv_fel_count	0.067	0.474
juv_misd_count	juv_misd_count	0.091	0.485
juv_other_count	juv_other_count	0.109	0.502
has_juv_felony	has_juv_felony	0.039	0.194
has_juv_misd	has_juv_misd	0.058	0.233
age_under_25	age_under_25	0.212	0.409
age_25_45	age_25_45	0.570	0.495

We can see that age with a sd of 11.889 has the largest variation, while native_american (the dummy variable for whether a observation is native american) a sd of .050 has the least variation.

Explore Key Predictors

```
compas$priors_cat <- NA  
  
compas$priors_cat[compas$priors_count == 0] <- "0"  
compas$priors_cat[compas$priors_count %in% 1:2] <- "1-2"  
compas$priors_cat[compas$priors_count %in% 3:5] <- "3-5"  
compas$priors_cat[compas$priors_count %in% 6:10] <- "6-10"
```

```

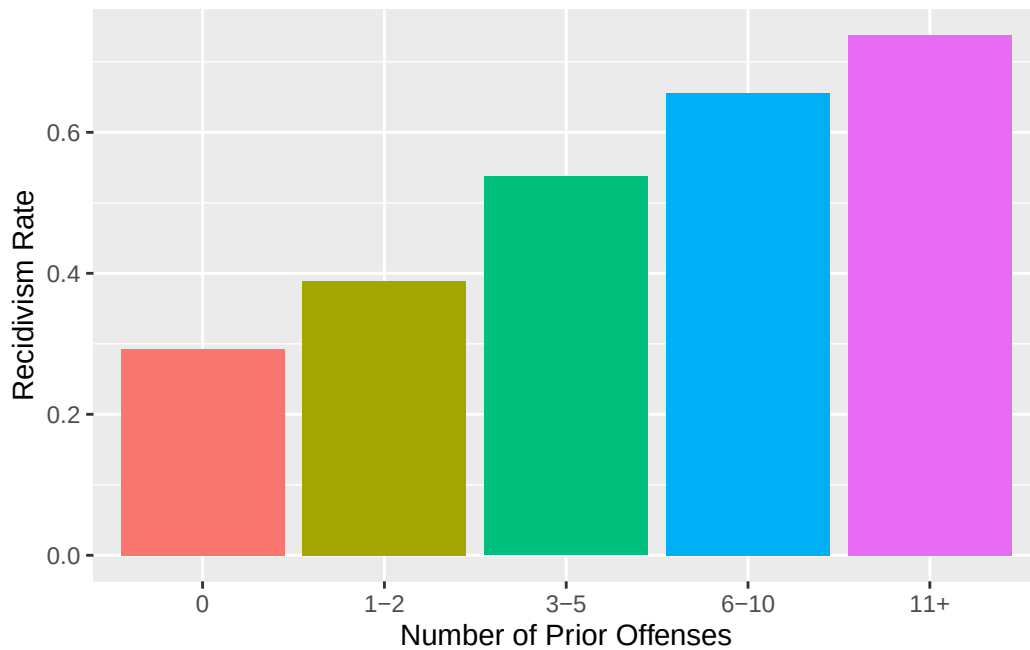
compas$priors_cat[compas$priors_count >= 11] <- "11+"

compas$priors_cat <- factor(compas$priors_cat,
                           levels = c("0", "1-2", "3-5", "6-10", "11+"))

rates <- compas %>%
  group_by(priors_cat) %>%
  summarise(rate = mean(recidivism, na.rm = TRUE))

ggplot(rates, aes(x = priors_cat, y = rate, fill = priors_cat)) +
  geom_bar(stat = "identity") +
  labs(x = "Number of Prior Offenses",
       y = "Recidivism Rate") +
  theme(legend.position = "none")

```



We can see that as the number of prior offenses increases, the recidivism rate increases.

2. OLS Baseline

Train/test Split

```
set.seed(51)
n <- nrow(compas)
train_idx <- sample(1:n, size = floor(0.8 * n))
train <- compas[train_idx, ]
test <- compas[-train_idx, ]

nrow(train)
```

```
[1] 5771
```

```
nrow(test)
```

```
[1] 1443
```

There are 5771 observations in the training set, and 1443 in the test set.

Fit OLS with Main Effects Only

```
set.seed(51)
train_nocat <- subset(train, select = -priors_cat)
model2.2 <- lm(recidivism ~ ., data = train_nocat)
summary(model2.2)
```

Call:

```
lm(formula = recidivism ~ ., data = train_nocat)
```

Residuals:

Min	1Q	Median	3Q	Max
-1.3347	-0.3966	-0.1892	0.4520	1.1865

Coefficients:

Estimate	Std. Error	t value	Pr(> t)
----------	------------	---------	----------

(Intercept)	0.732659	0.065429	11.198	< 2e-16	***
age	-0.010259	0.001169	-8.772	< 2e-16	***
female	-0.059808	0.015632	-3.826	0.000132	***
black	0.012266	0.014002	0.876	0.381049	
hispanic	-0.031099	0.023196	-1.341	0.180073	
asian	-0.035871	0.092911	-0.386	0.699452	
other_race	-0.046504	0.028482	-1.633	0.102577	
native_american	-0.020283	0.119810	-0.169	0.865572	
priors_count	0.029532	0.001401	21.072	< 2e-16	***
felony	0.037706	0.012938	2.914	0.003578	**
juv_fel_count	0.005821	0.017516	0.332	0.739647	
juv_misd_count	-0.028568	0.018950	-1.508	0.131723	
juv_other_count	0.023089	0.012614	1.830	0.067244	.
has_juv_felony	0.041631	0.044485	0.936	0.349395	
has_juv_misd	0.081995	0.039875	2.056	0.039802	*
age_under_25	-0.015671	0.040972	-0.382	0.702119	
age_25_45	-0.081454	0.028916	-2.817	0.004865	**

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4613 on 5754 degrees of freedom
 Multiple R-squared: 0.1429, Adjusted R-squared: 0.1405
 F-statistic: 59.95 on 16 and 5754 DF, p-value: < 2.2e-16

Variables with a significant at the 5% level: age, female, priors_count, felony, age_25_45, has_juv_misd.

A one-unit increase in the number of prior offenses is associated with a 0.0295 increase in the probability of recidivism, holding all other variables constant.

In-Sample vs Out-of-sample RMSE

```
set.seed(51)
yhat_train <- predict(model2.2, newdata = train)
rmse_train <- sqrt(mean((yhat_train - train$recidivism)^2))
rmse_train
```

```
[1] 0.4605811
```

```
yhat_test <- predict(model2.2, newdata = test)
rmse_test <- sqrt(mean((yhat_test - test$recidivism)^2))
rmse_test
```

```
[1] 0.462227
```

```
rmse_test - rmse_train
```

```
[1] 0.001645971
```

The in-sample RMSE: 0.4606

The out-of-sample RMSE: 0.4622

The gap is 0.0016, which is very small, indicating that the model is nearly identical for the training and test data, so there is no meaningful overfitting with 16 predictors.

Cross-Validation by Hand

```
set.seed(51)
folds <- sample(rep(1:5, length.out = nrow(train_nocat)))
rmse_cv <- numeric(5)

for (k in 1:5) {
  train_k <- train_nocat[folds != k, ]
  test_k <- train_nocat[folds == k, ]

  model_k <- lm(recidivism ~ ., data = train_k)
  pred_k <- predict(model_k, newdata = test_k)

  rmse_cv[k] <- sqrt(mean((test_k$recidivism - pred_k)^2))}

mean(rmse_cv)
```

```
[1] 0.4620144
```

A 5-fold CV RMSE of 0.4620 is extremely close to the single-split test RMSE, indicating very similar estimated out-of-sample performance.

Cross-validation is more reliable because it averages over multiple splits instead of using a single split that may not be representative. Using multiple splits therefore will reduce variability and give more stable estimates.

Cross-validation is correct for choosing λ because it minimizes prediction error. By evaluating many possible values, it balances bias and variance thus reducing overfitting.

3. The Kitchen Sink: What Happens when You Add Too Many Predictors?

Build the Expanded Model

```
train_nocat <- subset(train, select = -priors_cat)
test_nocat  <- subset(test,  select = -priors_cat)

f_full <- recidivism ~ (.)^2 +
  I(age^2) + I(priors_count^2) +
  I(juv_fel_count^2) + I(juv_misd_count^2) +
  I(juv_other_count^2)

X_train <- model.matrix(f_full, data = train_nocat)[, -1]
X_test  <- model.matrix(f_full, data = test_nocat)[, -1]

y_train <- train_nocat$recidivism
y_test  <- test_nocat$recidivism

ncol(X_train)
```

```
[1] 141
```

```
nrow(train_nocat)
```

```
[1] 5771
```

```
nrow(train_nocat) / ncol(X_train)
```

```
[1] 40.92908
```

The expanded model has 141 predictors.

Thus the ratio is 40.9 which means there are less observations per predictor increasing the chance of overfitting.

Fit OLS with the Kitchen Sink

```
ols_full <- lm(f_full, data = train_nocat)

pred_train <- predict(ols_full, newdata = train_nocat)
rmse_train <- sqrt(mean((train_nocat$recidivism - pred_train)^2))
rmse_train
```

```
[1] 0.4485474
```

```
pred_test <- predict(ols_full, newdata = test_nocat)
```

Warning in predict.lm(ols_full, newdata = test_nocat): prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases

```
rmse_test <- sqrt(mean((test_nocat$recidivism - pred_test)^2))
rmse_test
```

```
[1] 0.4890411
```

```
rmse_test - rmse_train
```

```
[1] 0.04049367
```

The in-sample RMSE is 0.4485

The out-of-sample RMSE is 0.4890

The difference is 0.0405

Compared to the simple model with a gap of about 0.0016, this gap is much larger. This expanded model is overfitting. With so many predictors, the model fits the training data better, which is why there is a lower in-sample RMSE, but performs worse on new data, leading to a higher out-of-sample RMSE. This happens because the model is capturing noise rather than true underlying patterns.

The Overfitting Lesson

Adding so many interaction terms increased the models flexibility, so it began to individually fit every data point instead of finding a good fit. This led to increased noise in the model. When we use a new data set, the model did not predict well because it was still using the noise from the training set. This is overfitting.

4. Ridge Regression: Shrink Everything

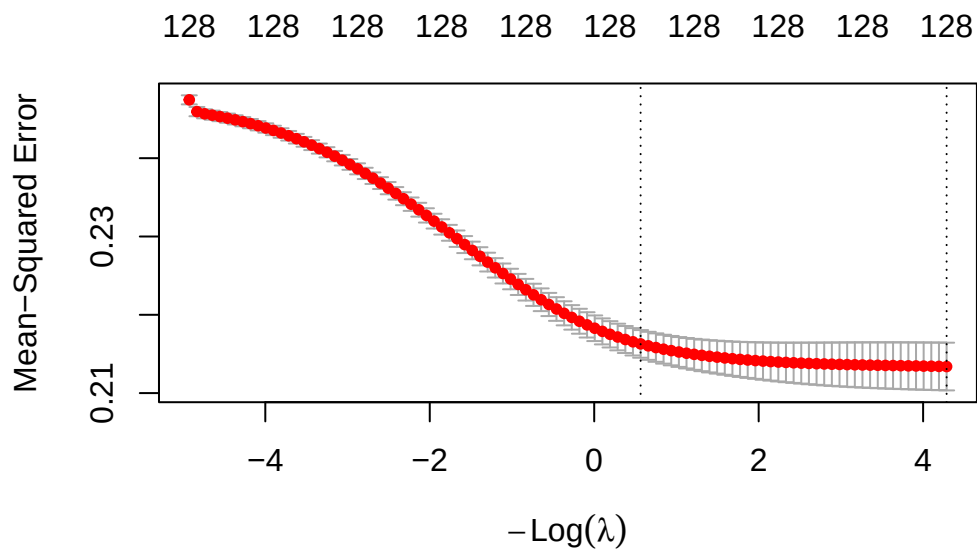
Fit Ridge with Cross-validation

```
set.seed(51) # for reproducible CV folds
cv_ridge <- cv.glmnet(X_train, y_train, alpha = 0, nfolds = 10)

cv_ridge$lambda.min
```

```
[1] 0.01376272
```

```
plot(cv_ridge)
```



The optimal lambda is 0.0138

From the graph we can see that prediction error decreases as λ increases until it reaches an optimal balance at the minimum before error flattens.

Ridge RMSE

```
pred_ridge <- predict(cv_ridge, s = "lambda.min", newx = X_test)
rmse_ridge <- sqrt(mean((y_test - pred_ridge)^2))
rmse_ridge
```

```
[1] 0.4700385
```

Simple OLS RMSE: 0.4622

Kitchen-Sink OLS RMSE: 0.489

Ridge RMSE: 0.470

The ridge regression improved to limit the overfitting from kitchen-sink by shrinking the coefficients thus reducing variance. It does not outperform the simple model though which indicates that the increase in interaction terms does not necessarily help.

Did Ridge set any Coefficients to Zero

```
coef(cv_ridge, s = "lambda.min")
```

```
142 x 1 sparse Matrix of class "dgCMatrix"
               lambda.min
(Intercept)    6.240172e-01
age           -7.146883e-03
female        -6.310304e-02
black          8.833220e-02
hispanic      -9.399273e-02
asian         -9.631129e-02
other_race     1.172025e-02
native_american -3.951980e-01
priors_count   2.814853e-02
felony         3.058693e-02
juv_fel_count  1.076914e-02
juv_misd_count 4.656344e-03
juv_other_count -1.444343e-03
has_juv_felony 1.081862e-01
has_juv_misd   9.650399e-02
age_under_25   8.001654e-02
```

age_25_45	-1.704785e-02
I(age^2)	-8.791274e-06
I(priors_count^2)	-7.997212e-04
I(juv_fel_count^2)	1.229990e-03
I(juv_misd_count^2)	-5.644604e-04
I(juv_other_count^2)	-9.162454e-03
age:female	-5.157715e-04
age:black	-1.586003e-03
age:hispanic	1.410240e-03
age:asian	-6.709189e-04
age:other_race	-9.495021e-04
age:native_american	-4.925668e-03
age:priors_count	2.275284e-04
age:felony	-9.447363e-04
age:juv_fel_count	-6.818760e-04
age:juv_misd_count	-8.816889e-04
age:juv_other_count	5.954111e-04
age:has_juv_felony	-3.010842e-03
age:has_juv_misd	1.839858e-03
age:age_under_25	-5.473185e-03
age:age_25_45	-2.113682e-03
female:black	-5.557765e-02
female:hispanic	1.584091e-02
female:asian	-5.213759e-02
female:other_race	1.963641e-02
female:native_american	5.948221e-01
female:priors_count	5.893477e-03
female:felony	5.244157e-02
female:juv_fel_count	1.769619e-01
female:juv_misd_count	4.550161e-02
female:juv_other_count	-3.235086e-02
female:has_juv_felony	-2.090400e-01
female:has_juv_misd	-1.424213e-01
female:age_under_25	-7.660951e-02
female:age_25_45	4.893753e-02
black:hispanic	.
black:asian	.
black:other_race	.
black:native_american	.
black:priors_count	2.533953e-04
black:felony	-2.439377e-03
black:juv_fel_count	5.469707e-03
black:juv_misd_count	-1.509481e-02

black:juv_other_count	7.799527e-02
black:has_juv_felony	2.720883e-02
black:has_juv_misd	-1.319078e-01
black:age_under_25	5.796655e-02
black:age_25_45	-4.565325e-02
hispanic:asian	.
hispanic:other_race	.
hispanic:native_american	.
hispanic:priors_count	-4.246163e-03
hispanic:felony	-4.338215e-03
hispanic:juv_fel_count	7.029226e-02
hispanic:juv_misd_count	1.920545e-01
hispanic:juv_other_count	-2.622615e-02
hispanic:has_juv_felony	1.554805e-01
hispanic:has_juv_misd	-5.198679e-01
hispanic:age_under_25	1.107656e-01
hispanic:age_25_45	1.912834e-02
asian:other_race	.
asian:native_american	.
asian:priors_count	1.070921e-02
asian:felony	3.956993e-01
asian:juv_fel_count	.
asian:juv_misd_count	1.513275e-01
asian:juv_other_count	5.299783e-01
asian:has_juv_felony	.
asian:has_juv_misd	1.545502e-01
asian:age_under_25	-3.048093e-01
asian:age_25_45	-3.728278e-01
other_race:native_american	.
other_race:priors_count	1.647782e-02
other_race:felony	1.394422e-02
other_race:juv_fel_count	-3.977307e-03
other_race:juv_misd_count	4.080779e-01
other_race:juv_other_count	-1.644768e-01
other_race:has_juv_felony	-5.716222e-01
other_race:has_juv_misd	-6.862896e-01
other_race:age_under_25	-3.777723e-02
other_race:age_25_45	-7.315482e-02
native_american:priors_count	2.116843e-02
native_american:felony	-2.371184e-02
native_american:juv_fel_count	5.667275e-02
native_american:juv_misd_count	2.968025e-01
native_american:juv_other_count	-3.425949e-02

native_american:has_juv_felony	4.365763e-01
native_american:has_juv_misd	2.862483e-01
native_american:age_under_25	2.747477e-01
native_american:age_25_45	1.892453e-01
priors_count:felony	1.193952e-04
priors_count:juv_fel_count	-1.179082e-03
priors_count:juv_misd_count	-3.065749e-04
priors_count:juv_other_count	1.076960e-03
priors_count:has_juv_felony	-5.526281e-03
priors_count:has_juv_misd	1.947317e-04
priors_count:age_under_25	2.603582e-02
priors_count:age_25_45	9.792224e-03
felony:juv_fel_count	2.748596e-02
felony:juv_misd_count	1.852399e-02
felony:juv_other_count	1.716179e-02
felony:has_juv_felony	-4.472484e-02
felony:has_juv_misd	-4.121938e-03
felony:age_under_25	-9.289630e-03
felony:age_25_45	3.091927e-02
juv_fel_count:juv_misd_count	-2.957597e-02
juv_fel_count:juv_other_count	3.162266e-02
juv_fel_count:has_juv_felony	1.345364e-02
juv_fel_count:has_juv_misd	-2.788563e-03
juv_fel_count:age_under_25	3.142700e-02
juv_fel_count:age_25_45	-1.221178e-02
juv_misd_count:juv_other_count	1.706586e-02
juv_misd_count:has_juv_felony	-2.571617e-02
juv_misd_count:has_juv_misd	6.921910e-03
juv_misd_count:age_under_25	-1.273201e-02
juv_misd_count:age_25_45	-8.842862e-03
juv_other_count:has_juv_felony	-6.655421e-02
juv_other_count:has_juv_misd	-3.578465e-02
juv_other_count:age_under_25	1.919057e-02
juv_other_count:age_25_45	-4.688677e-02
has_juv_felony:has_juv_misd	-1.885695e-03
has_juv_felony:age_under_25	-2.218784e-02
has_juv_felony:age_25_45	8.704239e-02
has_juv_misd:age_under_25	-6.167165e-03
has_juv_misd:age_25_45	5.805613e-02
age_under_25:age_25_45	.

No, Ridge did not eliminate any variables. Ridge shrinks coefficients towards zero but does

not set them exactly to zero. So, we can see that most coefficients are very small but not zero. The entries shown as “.” are not zero.

5. LASSO Regression: Shrink and Select

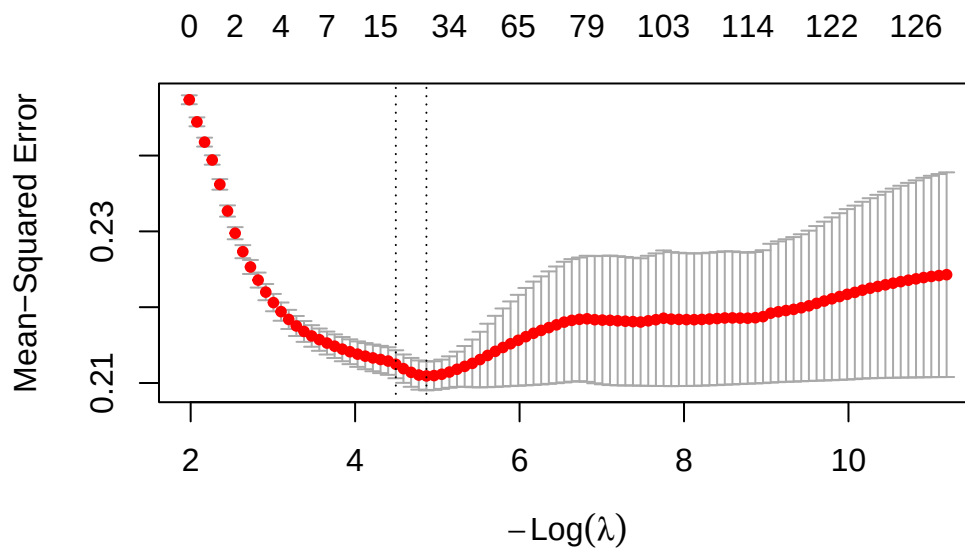
Fit LASSO with Cross-Validation

```
set.seed(51)
cv_lasso <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 10)

cv_lasso$lambda.min
```

```
[1] 0.007694477
```

```
plot(cv_lasso)
```



The optimal λ is 0.00769

LASSO Variable Selection

```
coef_lasso <- coef(cv_lasso, s = "lambda.min")

nonzero <- sum(coef_lasso != 0)
zero    <- sum(coef_lasso == 0)

nonzero
```

```
[1] 28
```

```
zero
```

```
[1] 114
```

Excluding the “intercept” there are 27 variables retained (the nonzero minus one). Thus, there are 114 eliminated.

```
coef_mat <- as.matrix(coef_lasso)
nonzero_vars <- rownames(coef_mat)[coef_mat != 0]
nonzero_vars
```

```
[1] "(Intercept)"      "age"
[3] "female"           "priors_count"
[5] "felony"           "I(priors_count^2)"
[7] "I(juv_other_count^2)" "age:female"
[9] "age:age_25_45"     "female:black"
[11] "female:priors_count" "female:age_under_25"
[13] "black:juv_fel_count" "black:juv_other_count"
[15] "black:age_under_25" "hispanic:juv_fel_count"
[17] "hispanic:age_25_45" "asian:age_25_45"
[19] "other_race:priors_count" "other_race:has_juv_felony"
[21] "other_race:age_25_45" "priors_count:age_under_25"
[23] "priors_count:age_25_45" "felony:has_juv_misd"
[25] "juv_fel_count:age_under_25" "juv_misd_count:has_juv_felony"
[27] "juv_other_count:age_under_25" "has_juv_felony:age_under_25"
```

Some surviving interaction terms: age:female, female:priors_counts, and many more. These make substantive sense as they suggest that the effect of one risk factor may depend on other characteristic rather than being the same for all.

LASSO reduces this subjective discretion because it selects variables on predictive performance than some choice of the researcher. So, instead of a human using their own intuition or biases to decide which interactions to include or not include, LASSO keeps the interactions that improve out-of sample prediction

LASSO RMSE

```
pred_lasso <- predict(cv_lasso, s = "lambda.min", newx = X_test)

rmse_lasso <- sqrt(mean((y_test - pred_lasso)^2))
rmse_lasso
```

```
[1] 0.4592597
```

Simple OLS RMSE: 0.4622

Kitchen-Sink OLS RMSE: 0.489

Ridge RMSE: 0.470

LASSO RMSE: 0.4593

Lasso performs the best for out-of-sample RMSE for all tests we have done.

6. Model Comparison and Interpretation

Elastic Net

```
set.seed(51)

cv_enet <- cv.glmnet(X_train, y_train, alpha = 0.5, nfolds = 10)

pred_enet <- predict(cv_enet, s = "lambda.min", newx = X_test)

rmse_enet <- sqrt(mean((y_test - pred_enet)^2))
rmse_enet
```

```
[1] 0.4583446
```

```
coef_enet <- as.matrix(coef(cv_enet, s = "lambda.min"))
sum(coef_enet != 0) - 1
```

```
[1] 43
```

Out-of-sample RMSE: 0.4583

Variables retained: 43

Big Comparison Table

```
comp_tb <- data.frame(Model = c("OLS (main effects, 16 vars)",
                                "OLS (kitchen sink, ~141 vars)",
                                "Ridge",
                                "LASSO",
                                "Elastic Net"),
                      Predictors = c(length(coef(model2.2)) - 1,
                                      ncol(X_train),
                                      ncol(X_train),
                                      sum(as.matrix(coef(cv_lasso,
                                                         s = "lambda.min")) != 0)
                                      - 1,
                                      sum(as.matrix(coef(cv_enet,
```

```

s = "lambda.min")) != 0)
- 1),
"Out RMSE" = c(rmse_test,
               sqrt(mean((test_nocat$recidivism -
                           predict(ols_full, newdata =
                                test_nocat))^2)),
               rmse_ridge,
               rmse_lasso,
               rmse_enet))

```

Warning in predict.lm(ols_full, newdata = test_nocat): prediction from rank-deficient fit; attr(*, "non-estim") has doubtful cases

```
kable(comp_tb, digits = 4)
```

Model	Predictors	Out.RMSE
OLS (main effects, 16 vars)	16	0.4890
OLS (kitchen sink, ~141 vars)	141	0.4890
Ridge	141	0.4700
LASSO	27	0.4593
Elastic Net	43	0.4583

Elastic Net has the lowest out-of-sample RMSE (0.4583), slightly better than LASSO (0.4593).

Kitchen sink had the best in-sample fit, but that is due to overfitting which led to a worse fit for out-sample data.

The table indicates there is a sweet spot between bias and variance. Completely eliminating bias (such as in the kitchen-sink model) had high variance which caused overfitting and very poor generalization. Meanwhile, models like the Ridge, LASSO, and Elastic Net saw increase in bias but a big a reduction in variance which improved the out-of-sample performance.

Baker Article Connection

Lasso helped to narrow the disagreement by selecting the relevant peer firms based on a predictive relationship with the outcome instead of either of the expert's (informed but not entirely unbiased) choice. This change helped to reduce the differences in the model's specification allowing the two to find more similar outcomes

The traditional OLS approach gives the most room for manipulation due to the ability to manually select which variables are included in the model. Instead of having the variables included or not included through some algorithm (LASSO), researchers can “turn on” or “turn off” specific variables on their own accord.

Prediction vs Explanation

The LASSO coefficient on `priors_count` would not answer any causal question as LASSO eliminates/chooses variables for prediction, not causal inference. Thus, LASSO does not account for confounding or omitted variable bias. The resulting estimated relationship shows correlation in the dataset not necessarily a causal effect.

Regression for prediction aims to minimize the out-of-sample error, allowing for more accurate predictions of outcomes, but it does not consider whether the relationships are causal. Meanwhile, regression for causal inference hopes to estimate how one variable affects another which requires much more careful strategies.

Algorithmic Fairness

```
pred_test <- predict(model2.2, newdata = test_nocat)
```

```
white <- test_nocat$black == 0 &  
  test_nocat$hispanic == 0 &  
  test_nocat$asian == 0 &  
  test_nocat$other_race == 0 &  
  test_nocat$native_american == 0
```

```
mean(pred_test[test_nocat$black == 1], na.rm = TRUE)
```

```
[1] 0.511985
```

```
mean(pred_test[white], na.rm = TRUE)
```

```
[1] 0.3789606
```

```
mean(test_nocat$recidivism[test_nocat$black == 1], na.rm = TRUE)
```

```
[1] 0.521262
```

```
mean(test_nocat$recidivism[white], na.rm = TRUE)
```

```
[1] 0.4020202
```

```
sum(test_nocat$black == 1)
```

```
[1] 729
```

```
sum(white)
```

```
[1] 495
```

Predicted Recidivism Rate: black defendants = 0.512

Predicted Recidivism Rate: white defendants = 0.379

Thus we can see there is a clear difference as the predicted risk of recidivism is higher for black defendants.

Actual Recidivism Rate: black defendants = 0.521

Actual Recidivism Rate: white defendants = 0.402

Actual rates of recidivism are higher for black defendants than white defendants, but both are underpredicted by the model.

Predictive algorithms can help in criminal justice by improving accuracy for rates like recidivism rates for a specific subsection of the population. However, you risk applying any skewness or bias from the model on the population. Moreover, underlying disparities may be exacerbated. While black recidivism rates are higher for black defendants, the predicted difference is much larger than the real difference.